۱- اتصال LCD به میکروکنترلر ATMEGA16

برنامه ای بنویسید که در سطر اول نمایشگر کلمه "Hi" و در سطر دوم کلمه "Hello" را به صورت کاراکتری نمایش دهد.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم.برای استفاده از LCD وارد تب Alphanumeric LCD می شویم و آن را Enable میکنیم. چون از LCD 16*2 استفاده میکنیم گزینه Characters/Line را برابر 16 میکنیم. ویزارد به صورت خودکار پورت A را به LCD اختصاص می دهد. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. از آنجایی که دستورات متناوب نداریم بنابراین نیازی نیست کد را در حلقه بی نهایت بنویسیم. در تابع main بعد از خط lcd_init(16) با دستور lcd_clear() صفحه نمایش را پاک میکنیم. باید به سطر اول بریم و کلمه "Hi" را بنویسیم بنابراین با استفاده از دستور lcd_gotoxy(0,0) مکان نمای LCD را به ستون اول و سطر اول میبریم. سپس با دستور lcd_puts("Hi") کلمه "Hi" را چاپ میکنیم. حال باید به خط دوم برویم پس lcd_gotoxy(0,1) را اضافه میکنیم و برای چاپ، دستور lcd_puts("Hello") را می نویسیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که صفحه نمایش را پاک میکند. به ستون اول از سطر اول نمایشگر می رود و کلمه "Hi" را می نویسد. سپس به ستون اول از سطر دوم رفته و کلمه "Hello" را می نویسد.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. LCD 16*2 (LM016L) و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. VSS را به زمین و VDD را به 5v+ متصل میکنیم. پایه VEE برای کنترل کنتراست کاراکتر ها استفاده می شود که میتوان با وصل کردن آن به پتانسیومتر مقدار آن را کنترل کرد. طبق سیم کشی ویزارد باید پایه های RS، RW و E را به PORTA0…2 وصل کنیم تا خود کدویژن عملیات کنترلی LCD را مدیریت کند. پایه های دیتا D0…3 را استفاده نمیکنیم و فقط پایه های D4…7 را به PORTA4…7 وصل میکنیم تا انتقال اطلاعات با کمک توابع کتابخانه ای کد ویژن صورت پذیرد. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که کلمه "Hi" در سطر اول و کلمه "Hello" در سطر دوم نمایشگر چاپ شده است.

۲- اتصال LCD به میکروکنترلر ATMEGA16

با استفاده از جدول برنامه ای بنویسید که در سطر اول نمایشگر کلمه "Hi" و در سطر دوم کلمه "Hello"را از طریق کد اسکی به نمایشگر ارسال و به صورت لاتین در نمایشگر نشان داده شود.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم.برای استفاده از LCD وارد تب Alphanumeric LCD می شویم و آن را Enable میکنیم. چون از LCD 16*2 استفاده میکنیم گزینه Characters/Line را برابر 16 میکنیم. ویزارد به صورت خودکار پورت A را به LCD اختصاص می دهد. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. از آنجایی که دستورات متناوب نداریم بنابراین نیازی نیست کد را در حلقه بی نهایت بنویسیم. در ابتدای تابع main متغیر i را برای استفاده در حلقه for و آرایه های Hi و Hello را جهت نگهداری کد هگز کاراکترهای کلمه مربوطه، تعریف میکنیم. کد اسکی حروف کلمات "Hi" و "Hello" را با کمک جدول در آرایه های Hi و Hello می نویسیم. در تابع main بعد از خط lcd_init(16) با دستور lcd_clear() صفحه نمایش را پاک میکنیم. باید به سطر اول بریم و کلمه "Hi" را بنویسیم بنابراین با استفاده از دستور lcd_gotoxy(0,0) مکان نمای LCD را به ستون اول و سطر اول میبریم. سپس حلقه for از i=0 تا i<2 را می نویسیم. در داخل حلقه دستور lcd_putchar(Hi[i]) را مینویسیم تا با اجرای حلقه، تک به تک حروف کلمه Hi چاپ شود. حال باید به خط دوم برویم پس lcd_gotoxy(0,1) را اضافه میکنیم و سپس حلقه for از i=0 تا i<5 را می نویسیم. در داخل حلقه دستور lcd_putchar(Hello[i]) را مینویسیم تا با اجرای حلقه، تک به تک حروف کلمه Hello چاپ شود. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که صفحه نمایش را پاک میکند. به ستون اول از سطر اول نمایشگر می رود و تک تک حروف کلمه "Hi" را با کمک کد اسکی چاپ میکند. سپس به ستون اول از سطر دوم رفته و کلمه "Hello" را به شکل مشابهی چاپ میکند.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. LCD 16*2 (LM016L) و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. VSS را به زمین و VDD را به 5v+ متصل میکنیم. پایه VEE برای کنترل کنتراست کاراکتر ها استفاده می شود که میتوان با وصل کردن آن به پتانسیومتر مقدار آن را کنترل کرد. طبق سیم کشی ویزارد باید پایه های RS، RW و E را به PORTA0…2 وصل کنیم تا خود کدویژن عملیات کنترلی LCD را مدیریت کند. پایه های دیتا D0…3 را استفاده نمیکنیم و فقط پایه های D4…7 را به PORTA4…7 وصل میکنیم تا انتقال اطلاعات با کمک توابع کتابخانه ای کد ویژن صورت پذیرد. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که کلمه "Hi" در سطر اول و کلمه "Hello" در سطر دوم نمایشگر چاپ شده است.

پیوست ها:

آزمایش ۱)

G:\Personal\Uni\Micro Lab\Micro\Az3-1\Az3-1.c

Notes | Az3-1.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session3-Exercise1
Version :
Date    : 10/7/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>

// Alphanumeric LCD functions
#include <alcd.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
```

G:\Personal\Uni\Micro Lab\Micro\Az3-1\Az3-1.c

Notes | Az3-1.c

```
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

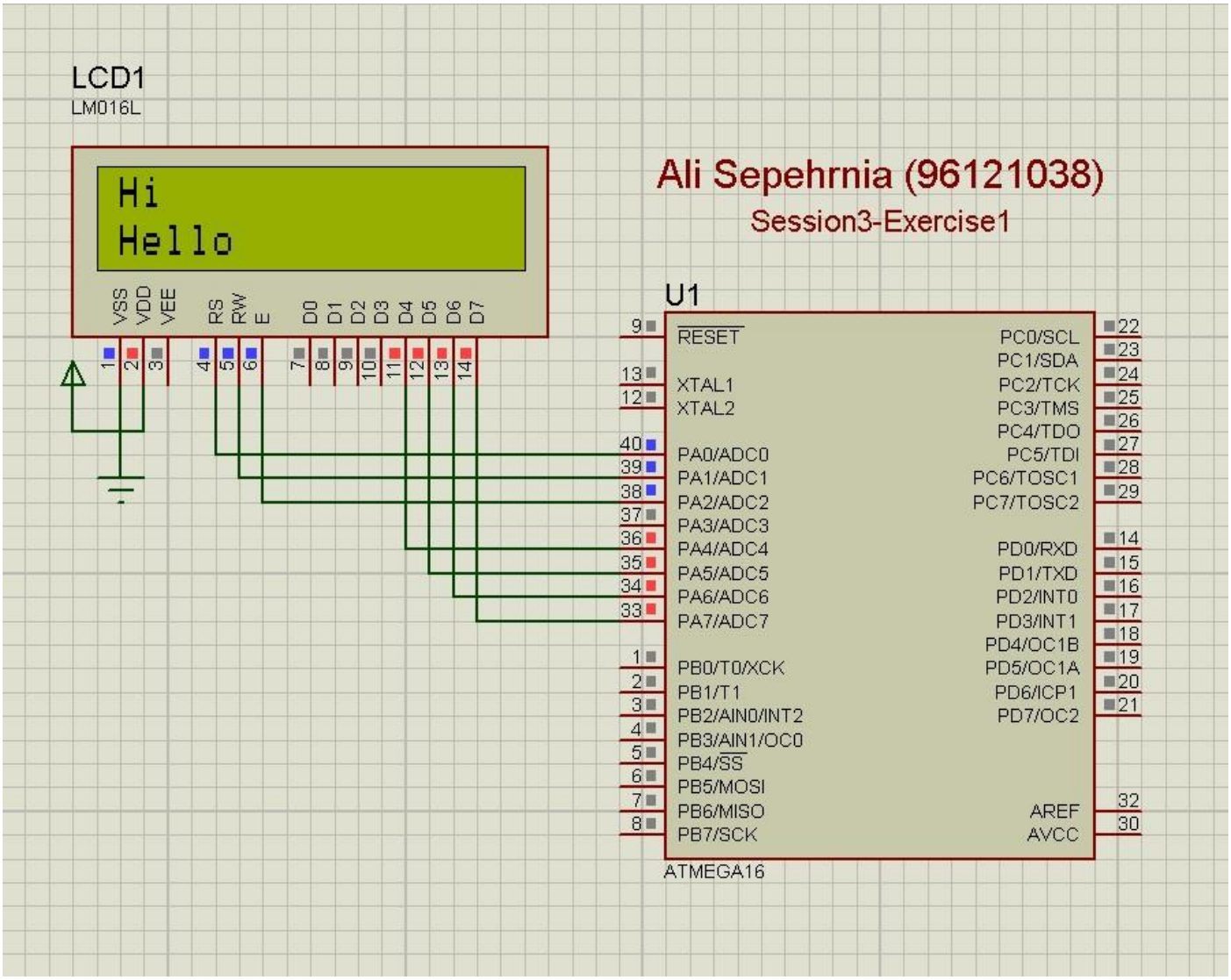
// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Alphanumeric LCD initialization
// Connections are specified in the
// Project|Configure|C Compiler|Libraries|Alphanumeric LCD menu:
// RS - PORTA Bit 0
// RD - PORTA Bit 1
// EN - PORTA Bit 2
// D4 - PORTA Bit 4
// D5 - PORTA Bit 5
// D6 - PORTA Bit 6
// D7 - PORTA Bit 7
// Characters/line: 16
```

G:\Personal\Uni\Micro Lab\Micro\Az3-1\Az3-1.c

Notes | Az3-1.c

```
71  lcd_init(16);
72  lcd_clear();
73  lcd_gotoxy(0,0);
74  lcd_puts("Hi");
75  lcd_gotoxy(0,1);
76  lcd_puts("Hello");
77
78  while (1)
79      {
80      // Place your code here
81
82      }
83  }
84
```



آزمایش ۲)

G:\Personal\Uni\Micro Lab\Micro\Az3-2\Az3-2.c

Notes | Az3-2.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session3-Exercise2
Version :
Date    : 10/17/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>

// Alphanumeric LCD functions
#include <alcd.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here
int i;
int Hi[] = {0x48,0x69};
```

G:\Personal\Uni\Micro Lab\Micro\Az3-2\Az3-2.c

Notes | Az3-2.c

```
int Hello[] = {0x48,0x65,0x6c,0x6c,0x6f};

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Alphanumeric LCD initialization
// Connections are specified in the
// Project|Configure|C Compiler|Libraries|Alphanumeric LCD menu:
// RS - PORTA Bit 0
// RD - PORTA Bit 1
// EN - PORTA Bit 2
// D4 - PORTA Bit 4
// D5 - PORTA Bit 5
```

G:\Personal\Uni\Micro Lab\Micro\Az3-2\Az3-2.c

Notes | Az3-2.c

```
// D6 - PORTA Bit 6
// D7 - PORTA Bit 7
// Characters/line: 16
lcd_init(16);
lcd_clear();
lcd_gotoxy(0,0);
for(i=0;i<2;i++)
{
    lcd_putchar(Hi[i]);
}
lcd_gotoxy(0,1);
for(i=0;i<5;i++)
{
    lcd_putchar(Hello[i]);
}

while (1)
      {
      // Place your code here

      }
}
```

